12. DNS Basics: Resolvers, Records, dig

12. DNS Basics: Resolvers, Records, dig

Phase: Phase 4 — DNS
Estimated time: 2 days
Slug: `12-dns-basics`

Learning objectives

- Query A/AAAA/CNAME/MX/TXT/NS
- Explain recursive vs authoritative
- Use dig/drill/nslookup
- Read TTL and caching implications

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

Browser fails, `ping` to an IP works. That is DNS until proven otherwise — resolvers, record types, TTL, and `dig` literacy cut hours off incidents.

Core concepts

What you must be able to explain:

- Query A/AAAA/CNAME/MX/TXT/NS
- Explain recursive vs authoritative
- Use dig/drill/nslookup
- Read TTL and caching implications

Mental model for `12-dns-basics` (exercise kind `dig_short`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
dig example.com A +short
dig example.com AAAA +short
dig example.com NS +short
```

Sample output:

```
93.184.216.34
2606:2800:220:1:248:1893:25c8:1946
a.iana-servers.net.
```

Interpretation: A/AAAA/NS answers show what clients may use. Empty AAAA is not always an error on IPv4-only paths.

Example 2

```
dig @1.1.1.1 example.com +noall +answer +stats
```

Sample output:

```
example.com. 300 IN A 93.184.216.34
;; Query time: 18 msec
;; SERVER: 1.1.1.1#53
```

Interpretation: Pinning `@resolver` removes 'which stub resolver?' ambiguity. Query time and SERVER lines matter when comparing sites.

Example 3

```
dig not-a-real-xyz.invalid A +noall +comments 2>/dev/null | head -20
```

Sample output:

```
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: ...
```

Interpretation: NXDOMAIN means the name does not exist at the queried view — different from timeout or SERVFAIL.

Mini-lab

1. Re-type every worked command for **DNS Basics: Resolvers, Records, dig** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson's tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function's return value without looking.

Failure modes

Symptom	Likely cause	Next check
Flushing caches randomly as first step	See common mistakes	Re-run lesson commands
Ignoring TTL when debugging ‘still ol…	See common mistakes	Re-run lesson commands
Using +trace against rate-limited net…	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for DNS Basics: Resolvers, Records, dig

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- Flushing caches randomly as first step
- Ignoring TTL when debugging ‘still old IP’
- Using +trace against rate-limited networks carelessly

Exercises

Work in `exercises/12-dns-basics/`. Read that folder’s `README.md` before coding.

```
cd exercises/12-dns-basics
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `dig_short_flags`, `dns_type_a`, `dns_type_mx`. Each function is a tiny retrieval drill for **DNS Basics: Resolvers, Records, dig** — return the exact tokens/flags/labels from the lesson. Keep the comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/12-dns-basics/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. dig for AAAA?
2. What is +short?
3. Recursive vs authoritative?

Further reading

- https://www.rfc-editor.org/rfc/rfc1035